

Part. 1

IPC

Inter-Process Communication

프로세스 간 통신

발표자 **Janghwan Kim (Harley)**

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증



- 01 전역변수 + 뮤텝스 / 세마포어
- 02 공유메모리 + 링버퍼 (메시지 큐)
- 03 TCP / IP 소켓

목표

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증

01

쓰레드 / 프로세스 개념 및 통신 방법

개념

리눅스 환경에서 프로그램 · 프로세스 · 쓰레드가 무엇이고, 서로 어떻게 데이터를 주고받는지 정리한다.

02

IPC 개념 및 뮤텍스 / 세마포어

개념

UDP · TCP/IP · MessageQ · 링버퍼 · 공유메모리의 특징과 사용 상황, 장 / 단점. 그리고 이를 안전하게 묶어주는 동기화 수단.

03

데이터 송 / 수신 기능 구현 및 코드 정리

구현

송신 · 수신 쓰레드를 구성해 1 부터 100 까지 0.1 초 간격으로 주고받는다.
ToDo#1 쓰레드 간 · ToDo#2 프로세스 간 · ToDo#3 TCP / IP

오늘 발표는 1 → 2 → 3 순서로, 개념을 먼저 정리하고 그 개념이 코드 어디에 쓰였는지 이어서 보여드립니다.

01

쓰레드 / 프로세스 개념 및 통신 방법

리눅스 환경 기준

1-1

프로그램 · 프로세스 · 쓰레드

무엇이 다르고, 왜 그 차이 때문에 통신 방법이 갈리는가

1-2

통신 방법

쓰레드 간 · 프로세스 간에 쓸 수 있는 수단과 선택 기준

프로그램 · 프로세스 · 쓰레드

무엇을 공유하고 무엇을 따로 갖는지가 통신 방법을 결정한다

1 프로그램 Program

디스크에 저장된 실행 파일 . 아직 실행되지 않은 명령어와 데이터의 묶음이다 .

▸ 정적 — 메모리도 자원도 할당되지 않은 상태

2 프로세스 Process

실행 중인 프로그램 . 커널이 독립된 가상 주소공간과 자원 (PID · Process ID, 파일 디스크립터) 을 붙여준 단위다 .

▸ 서로의 메모리를 볼 수 없다 → 그래서 IPC 가 필요하다

3 쓰레드 Thread

프로세스 안의 실행 흐름 . Code · Data · Heap 은 공유하고 Stack 과 레지스터 (PC : Program Counter · SP : Stack Pointer) 만 따로 갖는다 .

▸ 전역변수로 바로 주고받는다 → 대신 동기화가 필수다

리눅스에서는 프로세스도 쓰레드도 커널 입장에선 task_struct 하나 . **clone()** 에 무엇을 공유할지만 다르게 준다 .

프로세스 A (독립된 가상 주소공간)

쓰레드끼리 공유하는 영역

Code

Data (전역변수)

Heap

파일 디스크립터

쓰레드 1 (송신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

쓰레드 2 (수신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

전역변수를 통해 복사 없이 바로 전달 · 단 , 동시 접근은 뮤텁스로 보호



IPC 필요

프로세스 B

주소공간이 다르다 → 직접 접근 불가

통신 방법

주소공간을 공유하느냐, 그리고 얼마나 멀리 보내느냐가 쓸 수 있는 수단을 정한다

① 같은 프로세스 안

쓰레드 ↔ 쓰레드 · 주소공간을 공유한다

전역변수 · 공유 자료구조

+ **뮤텍스 · 세마포어 · 조건변수**

주소만 넘기고 데이터는 옮기지 않는다

- 커널도 안 거치고 복사도 없어 가장 빠르다
- 동기화를 빠뜨리면 데이터가 깨지거나 데드락이 난다
- 한 쓰레드가 죽으면 프로세스 전체가 같이 죽는다

lock / unlock · sem_wait / sem_post

ToDo#1 **lpcThread.c**

② 같은 PC, 다른 프로세스

프로세스 ↔ 프로세스 · 커널 통로를 거친다

메시지 큐 · 공유메모리

+ **뮤텍스 · 세마포어 (직접 구현)**

참고 : 파이프 · FIFO · 시그널

- 커널이 메시지 경계와 순서를 지켜 준다
- 공유메모리는 복사가 없고, 메시지 큐는 두 번 복사한다
- 이름만 같으면 기동 순서는 상관없다

msgsnd / msgrcv · shmget · mmap

ToDo#2 **lpcMsgQ.c**

③ 다른 PC · 다른 프로그램

장비 ↔ 장비 · 네트워크를 건넌다

소켓 (TCP / UDP)

+ **연결 · 유실 · 순서를 직접 처리**

IP 와 포트만 알면 붙는다

- 유일하게 다른 장비까지 확장된다
- 헤더와 왕복 지연이 붙어 가장 느리다
- 상대가 리눅스여도 같은 방식으로 붙는다

socket / connect · send / recv

ToDo#3 **lpcSocket.c**

고르는 기준은 하나 — 얼마나 멀리 보내야 하는가

가까울수록 빠르다. 그만큼 커널이 해 주던 경계와 순서, 동기화가 내 코드로 넘어온다.

본 과제는 이 세 칸을 그대로 **ToDo#1 · ToDo#2 · ToDo#3** 으로 구현했다

02

IPC 개념 및 뮤텍스 / 세마포어

각 수단의 특징 · 어떤 상황에서 쓰는지 · 장 / 단점

2-1

UDP · TCP / IP

네트워크를 건너는 통신 · 신뢰성이냐 속도냐

2-2

**MessageQ · 링버퍼 ·
공유메모리**

같은 PC 안에서 데이터를 옮기는 세 가지 도구

2-3

뮤텍스 · 세마포어

공유 자원을 안전하게 만들어 주는 동기화 수단

2-1

UDP · TCP / IP

둘 다 IP 위에 올라간다. 차이는 하나 — 신뢰성을 커널이 책임지느냐, 앱이 책임지느냐

구분	TCP / IP Transmission Control Protocol / Internet Protocol	UDP User Datagram Protocol
연결	3-way handshake 로 맺고 시작한다	없음 — 주소만 알면 바로 보낸다
신뢰성	커널이 보장 (재전송 · 순서 · 중복)	보장 없음 — 앱이 직접 처리
데이터 단위	바이트 스트림 — 경계가 없다	데이터그램 — 보낸 크기 그대로
통신 상대	1 : 1 만 가능	1 : N 브로드캐스트 · 멀티캐스트
헤더 · 지연	20 byte · 재전송으로 지연이 튜다	8 byte · 가볍고 지연이 낮다
쓰는 곳	파일 전송 · 명령 / 제어 · 로그 수집	센서 · 음성 / 영상 스트리밍
과제 적용	ToDo#3 의 데이터 송 / 수신	TCP Sync 라인체크에만 사용

send 한 번이 recv 한 번으로 오지 않는다

송신

```
send(sock, buf, 4, 0)
4 byte 를 한 번에 보낸다
```

>

커널 TCP 버퍼

바이트로만 쌓인다 —
메시지 경계가 사라진다

>

수신 ①

```
recv(sock, buf, 4, 0)
이렇게 불러도 2 byte 만 올 수 있다
```

>

수신 ②

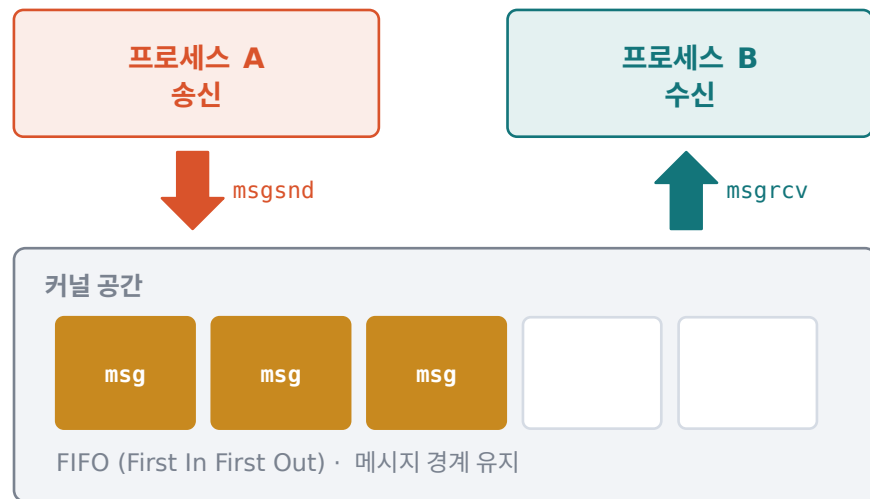
```
recv(sock, buf+2, 2, 0)
남은 2 byte 를 마저 받는다
```

→ 요청한 크기를 다 채울 때까지 반복 수신한다 · f_SocketRecvTCP_IPv4 가 그 루프다

메시지 큐 Message Queue

커널이 메시지 단위로 보관해 주는 큐. 보내는 쪽과 받는 쪽이 동시에 떠 있지 않아도 된다

메시지 큐 — 커널이 중간에서 보관한다



★ **Windows** 에는 **System V** 메시지 큐가 없다
→ ToDo#2 에서 공유메모리 + 링버퍼로 직접 구현했다

LINUX API (Application Programming Interface)

```
msgget(key, IPC_CREAT) · msgsnd(qid, &msg, size, 0)
msgrcv(qid, &msg, size, mtype, 0) · msgctl(IPC_RMID)
POSIX (Portable Operating System Interface) : mq_open / mq_send
/ mq_receive (우선순위 지원)
```

특징

- 커널 공간에 큐가 만들어지고, 프로세스들은 키 (key) 로 같은 큐를 찾아 붙는다
- 바이트가 아니라 메시지 단위라서 보낸 크기 그대로 받는다
- mtype(메시지 타입) 으로 원하는 종류만 골라 받을 수 있다
- 큐가 비면 받는 쪽이, 가득 차면 보내는 쪽이 커널에서 대기한다

어떤 상황에서 쓰는가

- 명령이나 이벤트처럼 한 건씩 주고받는 데이터
- 보내는 쪽과 받는 쪽 속도가 달라 중간에 쌓아둬야 할 때
- 어느 쪽이 먼저 떠도 상관없어야 할 때

장점

- 커널이 대기까지 해 줘서 따로 락이 필요 없다
- 메시지 경계와 순서가 보장된다
- 상대가 아직 안 떠 있어도 미리 넣어둘 수 있다
- 원하는 타입만 골라서 받을 수 있다

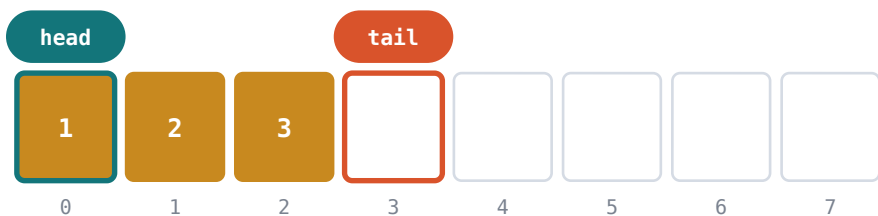
단점

- 유저와 커널 사이를 두 번 복사해 대용량에 불리하다
- 큐에 담기는 양이 정해져 있다 (기본 16KB)
- 지우지 않으면 커널에 계속 남는다
- 같은 PC 안에서만 동작한다

링버퍼 Ring Buffer

고정 크기 배열을 원형으로 돌려 쓰는 자료구조 . IPC 수단이 아니라 데이터를 쌓아 두는 방법이다

고정 크기 배열 8 칸을 원형처럼 돌려 쓴다



끝에 닿으면 $(tail + 1) \% N$ 으로 0 번 슬롯으로 되돌아간다

head — 읽는 위치
수신 쪽이 여기서 꺼낸다

tail — 쓰는 위치
송신 쪽이 여기에 넣는다

```
ring[tail] = msg;  tail = (tail+1) % N;  count++;
msg = ring[head]; head = (head+1) % N;  count--;
```

본 과제 적용

쓰레드 간 8 슬롯 전역 링버퍼 (lpcThread.c)

프로세스 간 64 슬롯 링버퍼를 공유메모리 위에 (lpcMsgQ.c)

특징

- 고정 크기 배열 + 읽는 위치 (head) / 쓰는 위치 (tail) 두 개의 인덱스
- 끝에 닿으면 나머지 연산으로 처음으로 되돌아간다 $\rightarrow tail = (tail + 1) \% N$
- 개수가 늘어도 넣고 빼는 시간이 그대로다 ($O(1)$), 실행 중 동적 할당도 없다
- 비었는지 찼는지 구별하려면 개수를 세거나 한 칸을 비워 둔다

어떤 상황에서 쓰는가

- 넣는 쪽과 꺼내는 쪽 속도가 다를 때 사이에 두고
- 시리얼 · 네트워크 수신 버퍼, 오디오 · 센서 스트림, 로그 버퍼
- 실행 중에 메모리를 할당하면 안 되는 실시간 코드

장점

- 넣기 / 빼기 시간이 개수와 무관하게 일정 ($O(1)$)
- 메모리 고정 — 단편화도 할당 지연도 없다
- 연속 메모리라 캐시에 유리하다
- 공유메모리 위에 그대로 올릴 수 있다

단점

- 크기가 고정이라 가득 차면 기다리거나 버려야 한다
- head · tail 갱신 자체가 임계구역이다
- 동기화를 직접 붙여야 한다
- 가변 길이 데이터는 담기 번거롭다

공유메모리 Shared Memory

두 프로세스가 각자의 주소공간에 같은 물리 메모리를 붙인다. 붙고 나면 그냥 포인터로 읽고 쓴다

같은 물리 메모리를 두 주소공간에 매핑

프로세스 A

가상 주소공간

p = 0x7F1A ...

프로세스 B

가상 주소공간

p = 0x5C40 ...

② mmap / MapViewOfFile
같은 함수인데 주소가 다르다

물리 메모리 (같은 페이지)

양쪽이 이 메모리를 직접 읽고 쓴다 · 옮겨 담지 않는다

※ 순서를 지켜 주는 장치는 여기에 없다

공유메모리 + 뮤텍스 / 세마포어 는 항상 한 세트다

API — ① 메모리를 만든다 ② 내 주소공간에 붙인다

Linux ① shmget / shm_open ② shmat / mmap
Windows ① CreateFileMapping ② MapViewOfFile
해제 : shmdt / shmctl UnmapViewOfFile / CloseHandle

특징

- 붙일 때만 커널을 거치고, 그 뒤 읽고 쓸 때는 시스템 콜이 없다
- 데이터를 옮겨 담지 않으니 IPC 수단 중 가장 빠르다
- 붙이면 내 주소공간의 주소가 돌아오는데 프로세스마다 다르다 → 포인터 대신 오프셋을 쓴다
- 컴파일러가 끼워 넣는 빈 공간 (패딩) 까지 양쪽이 똑같아야 한다 (#pragma pack)

어떤 상황에서 쓰는가

- 영상 프레임처럼 크고 자주 오가는 데이터
- 한 번 오갈 때 지연이 짧아야 하는 처리
- 여러 프로세스가 같은 데이터를 동시에 읽어야 할 때

장점

- 복사도 시스템 콜도 없어서 가장 빠르다
- 주고받는 크기에 제한이 거의 없다
- 자료구조를 그대로 올릴 수 있다
- 한 번 붙으면 접근 비용이 일정하다

단점

- 동기화 수단이 없어서 직접 붙여야 한다
- 한쪽 실수가 상대 메모리까지 깨뜨린다
- 포인터를 저장하면 안 된다 (주소가 다름)
- 누가 아직 쓰는지 몰라 정리 시점을 잡기 어렵다

세 가지 비교 — 그리고 조합

메시지 큐는 커널이 주는 기능, 링버퍼는 직접 짜는 자료구조, 공유메모리는 그 링버퍼를 올릴 메모리다

구분	메시지 큐	링버퍼	공유메모리
무엇인가	커널이 관리하는 IPC 객체	자료구조 (IPC 수단 아님)	커널이 연결해 주는 메모리
데이터 단위	메시지 1 개 — 넣은 만큼 나온다	슬롯 1 칸 — 크기 고정	바이트 — 경계는 내가 만든다
복사 횟수	2 회 — 내 변수 → 커널 → 상대	1 회 — 내 변수 → 슬롯	0 회 — 옮겨 담는 일이 없다
동기화	커널이 내장	직접 구현	직접 구현 (필수)
속도	보통	빠름	가장 빠름
용량	커널 메모리라 한도가 있다 (16KB)	고정 — 배열 크기가 곧 용량	물리 메모리가 허용하는 만큼
대표 용도	명령 · 이벤트 전달	들어오는 데이터 임시 보관	대용량 데이터 공유

메시지 큐가 없으면 — 이 넷으로 직접 만든다

세마포어 · 뮤텝스 = 표 동기화 행의 직접 구현 — 다음 장 2-3 에서 설명

공유메모리

두 프로세스가 같은
메모리를 함께 본다

+

링버퍼

FIFO 규칙 — 넣는 순서대로
꺼내지게 만든다

+

세마포어 2 개

빈 슬롯 · 찬 슬롯 개수
비면 · 차면 대기시킨다

+

뮤텝스

head · tail 갱신을
한 번에 하나만

= 메시지 큐 → ToDo#2 가 정확히 이 조합이다

무텍스 Mutex (MUTual EXclusion)

상호배제 — 임계구역에 한 번에 하나만 들어가게 하는 자물쇠

임계구역 — 한 번에 한 명만

쓰레드 1

lock 성공 → 진입



쓰레드 2

lock 대기 (블록)



임계구역 Critical Section

공유 데이터를 고치는 코드 — 동시에 들어오면 깨진다

MUTEX 잠금

기본 형태 — 실제 함수 이름은 아래 API

```
lock(m); /* 임계구역 : 공유 데이터 갱신 */ unlock(m);
```

API — 같은 프로세스 안이나, 프로세스 사이냐로 갈린다

Linux pthread_mutex_init → lock / unlock → destroy

Windows ① CRITICAL_SECTION — 한 프로세스 안, 유저 모드라 빠르다

InitializeCriticalSection → EnterCriticalSection / LeaveCriticalSection

Windows ② Named Mutex — 프로세스 사이, 이름으로 커널이 찾아 준다

CreateMutex(이름) → WaitForSingleObject / ReleaseMutex

특징

- 상태가 잠김과 풀림 두 가지뿐이다
- 소유권이 있다 — 잠근 쓰레드만 풀 수 있다
- 락을 못 잡으면 커널이 재워 두었다가 풀릴 때 깨운다 (CPU 를 쓰지 않는다)
- 한 프로세스 안이면 CRITICAL_SECTION, 프로세스 사이면 이름 붙인 Mutex 를 쓴다

어떤 상황에서 쓰는가

- 여러 쓰레드가 같은 변수나 자료구조를 고칠 때
- 링버퍼의 head · tail 인덱스 갱신, 링크드 리스트 · 해시 테이블

장점

- 쓰임이 단순해서 실수가 적고 디버깅이 쉽다
- 누가 잡고 있는지 분명해 데드락을 추적할 수 있다

단점

- 잡은 쪽만 풀 수 있어 신호 전달에는 못 쓴다
- 락 잡는 순서가 엇갈리면 데드락이 난다
- 조건을 기다리는 데 쓰면 잠갔다 풀었다 확인을 반복해 CPU 를 태운다

세마포어 Semaphore

쓸 수 있는 자원이 몇 개 남았는지 세는 카운터 . wait 로 하나 가져가고 post 로 하나 돌려준다

카운터가 자원의 개수를 센다

count = 3

wait (P)

하나 가져간다
count = 0 이면 대기

post (V)

하나 돌려준다
대기 중인 쪽을 깨움

자원 슬롯

사용 가능

사용 가능

사용 가능

사용 중

사용 중

```
sem_wait(&sem); /* 자원 사용 구간 */ sem_post(&sem);
```

API

Linux : sem_init / sem_wait / sem_post (POSIX)
semget / semop (System V)
Windows : CreateSemaphore(이름) 으로 만들고
WaitForSingleObject / ReleaseSemaphore

특징

- 0 이상의 정수 카운터 . wait(P) 는 1 감소하고 0 이면 대기 , post(V) 는 1 증가하고 대기자를 깨운다 (P · V 는 네덜란드어 검사 · 증가의 첫 글자)
- 소유권이 없어서 A 가 내리고 B 가 올려도 된다
- 0 이면 커널이 재워 두었다가 post 때 깨운다 (기다리는 동안 CPU 를 쓰지 않는다)

어떤 상황에서 쓰는가

- 커넥션 풀이나 버퍼 슬롯처럼 개수가 정해진 자원
- 한쪽이 채우고 다른 쪽이 꺼낼 때 찻다 비었다를 알림

장점

- 개수를 셀 수 있다 (뮤텍스로는 안 된다)
- 조건을 기다리는 일을 확인 반복 없이 할 수 있다
- 소유권이 없어 다른 주체가 신호를 줄 수 있다
- 이름을 주면 프로세스 간에도 쓸 수 있다

단점

- 소유자가 없어 잘못 쓴 지점을 찾기 어렵다
- post 를 한 번 빠뜨리면 영원히 기다린다

2-3

뉡덱스 vs 세마포어 — 왜 둘 다 쓰는가

상호배제는 뉡덱스 , 개수를 세고 기다리는 것은 세마포어 . 하나로 다른 하나를 흥내내면 반드시 문제가 생긴다

구분	뉡덱스	세마포어
값	잠김 / 풀림 (0 · 1)	0 이상의 정수 카운터
소유권	있다 — 잠근 쪽만 해제	없다 — 누구나 post 가능
목적	상호배제 (데이터 보호)	신호 · 개수 관리 (흐름 제어)
던지는 질문	지금 나만 만지고 있나 ?	쓸 수 있는 게 몇 개 남았나 ?
대표 용도	임계구역 보호	송신 - 수신 버퍼 , 자원 풀

뉡덱스만 쓰면
 ✕ 버퍼가 빌 때까지 기다리려면 락을 잡았다 풀었다 확인을 반복 → 바쁜 대기로 CPU 를 태운다

세마포어만 쓰면
 ✕ 개수는 맞지만 head · tail 을 두 쓰레드가 동시에 고쳐 데이터가 깨진다

둘을 같이 쓰면
 ✓ 세마포어로 기다리고 , 뉡덱스로 보호한다 → 바쁜 대기도 없고 데이터도 안전하다

송신 - 수신 표준 순서

송신

sem_wait(empty)

>

lock(mutex)

>

버퍼에 write

>

unlock(mutex)

>

sem_post(full)

수신

sem_wait(full)

>

lock(mutex)

>

버퍼에서 read

>

unlock(mutex)

>

sem_post(empty)

순서를 뒤집으면 데드락 — 뉡덱스를 먼저 잡고 세마포어를 기다리면 , 상대는 그 락을 영원히 잡지 못한다 .

03

데이터 송 / 수신 기능 구현 및 코드 정리

개발 환경

Windows · Visual Studio 2022 (v143) · x64

개발언어 C — IPC 코어는 전부 C DLL (Dynamic Link Library), 확인용 UI 만 MFC

공통 데이터 시나리오 (세 ToDo 모두 동일)

송신측 1 로 초기화된 int 를 0.1 초 간격으로 송신 , 이후 1 씩 증가 (100 까지)

수신측 받은 데이터를 그대로 출력

ToDo#1

쓰레드 간 메시지 송 / 수신

전역변수 + 뮤텍스 / 세마포어

`IpcThread.c`

ToDo#2

프로세스 간 메시지 송 / 수신

메시지 큐 (공유메모리 + 링버퍼)

`IpcMsgQ.c`

ToDo#3

다른 프로그램과 메시지 송 / 수신

TCP / IP (SocketUtility 포팅)

`IpcSocket.c`

IPC 로직은 전부 `IpcCore.dll` 안에 있고 , MFC (Microsoft Foundation Class) UI (User Interface) 는 버튼으로 코어 함수를 호출하고 로그만 표시한다 .

ToDo#1

쓰레드 간 송 / 수신 구현

전역 링버퍼를 뮤텁스로 보호하고, 빈 슬롯 · 찬 슬롯 개수를 세마포어 2 개로 센다 · lpcThread.c



LINUX 대응

CRITICAL_SECTION	→ pthread_mutex_t
CreateSemaphore / WaitForSingleObject	→ sem_init / sem_wait
ReleaseSemaphore	→ sem_post
_beginthreadex	→ pthread_create

※ CreateThread 대신 _beginthreadex — CRT (C Runtime) 자원이 정리되도록

f_IpcThreadQueueSend()

```
// ① 빈 슬롯 대기 — 가득 차면 여기서 블록
if (WaitForSingleObject(s_hSemEmpty, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 뮤텁스로 링버퍼 상호배제
EnterCriticalSection(&s_stRingLock);
s_staRing[s_iRingTail] = *stpMsg;
s_iRingTail = (s_iRingTail + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount++;
LeaveCriticalSection(&s_stRingLock);

// ③ 찬 슬롯 +1 — 수신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemFull, 1, NULL);
```

f_IpcThreadQueueRecv()

```
// ① 찬 슬롯 대기 — 비어 있으면 여기서 블록
if (WaitForSingleObject(s_hSemFull, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 같은 뮤텁스로 상호배제
EnterCriticalSection(&s_stRingLock);
*stpMsg = s_staRing[s_iRingHead];
s_iRingHead = (s_iRingHead + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount--;
LeaveCriticalSection(&s_stRingLock);

// ③ 빈 슬롯 +1 — 송신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemEmpty, 1, NULL);
```


실행 결과

한 프로세스 안에서 [Start] — 송신과 수신이 0.1 초 간격으로 교대로 찍힌다

● lpcUI — 로그창 · Thread [Start] → [Stop]

```
14:20:31 [TH] start
14:20:31 [TH] tx 1
14:20:31 [TH] rx 1
14:20:31 [TH] tx 2
14:20:31 [TH] rx 2
14:20:31 [TH] tx 3
14:20:31 [TH] rx 3
14:20:32 [TH] tx 4
14:20:32 [TH] rx 4
14:20:32 [TH] tx 5
14:20:32 [TH] rx 5
...
14:20:41 [TH] tx 98
14:20:41 [TH] rx 98
14:20:41 [TH] tx 99
14:20:41 [TH] rx 99
14:20:41 [TH] tx 100
14:20:41 [TH] rx 100
14:20:41 [TH] tx done (100)
14:20:46 [TH] rx done (100, seq err 0)
14:20:46 [TH] stop
```

무엇을 확인했나

1 1 부터 100 까지 빠짐없이

송신 100 건, 수신 100 건. 순서가 뒤바뀌거나 빠진 값이 없다.

2 tx 와 rx 가 교대로

링버퍼가 8 칸뿐이라 수신이 밀리면 송신이 세마포어에서 대기한다 — 흐름 제어가 동작한다는 뜻이다.

3 대기 중에도 [Stop] 에 즉시 반응

수신 쓰레드는 100ms 단위로 타임아웃을 두고 대기하므로 정지 요청이 바로 먹힌다.

실행 방법

lpcUI.exe 를 실행하고 Thread 영역의 [Start] 버튼 하나만 누르면 된다.
한 프로세스 안에서 송신 쓰레드와 수신 쓰레드가 한 쌍으로 뜬다.

ToDo#2

메시지 큐 구현

리눅스 환경의 메시지 큐가 윈도우 환경에는 없어서 공유메모리 + 링버퍼로 메시지 큐 구현 · IpcMsgQ.c

리눅스 메시지 큐 API ↔ 이 과제의 Windows 구현

Linux (System V)

본 과제의 Windows 구현

<code>msgget(key, IPC_CREAT)</code>	CreateFileMapping — 이름 붙인 공유메모리 + 링버퍼 헤더 초기화
<code>msgsnd()</code>	세마포어 (빈슬롯) → 뮅텍스 → write → 세마포어 (찬슬롯) +1
<code>msgrcv()</code>	세마포어 (찬슬롯) → 뮅텍스 → read → 세마포어 (빈슬롯) +1
<code>msgctl(IPC_RMID)</code>	모든 핸들 CloseHandle — 마지막 참조가 사라지면 정리된다
<code>mtpe</code>	ST_IpcMsg.iMsgType — 메시지 종류를 구분하는 필드

※ 이름 하나로 같은 큐를 찾아 붙고, 가득 차면 보내는 쪽이 · 비어 있으면 받는 쪽이 대기한다

큐 하나 = 공유메모리 + 링버퍼 + 세마포어 2 개 + 뮅텍스

두 프로세스는 서로의 메모리를 못 본다 — 같은 이름을 커널에 대면 같은 물건을 받는다

이름은 모두 `Local\IpcProc.IpcDemoQ` 로 시작하고 뒤에 넷이 붙는다

<code>.map</code>	공유메모리 — 링버퍼 64 슬롯이 여기 올라간다
<code>.mtx</code>	뮅텍스 — head · tail 을 한 번에 하나만
<code>.sem.e</code>	세마포어 — 빈 슬롯 수 (처음 64)
<code>.sem.f</code>	세마포어 — 찬 슬롯 수 (처음 0)

※ Local\ 이라 관리자 권한이 필요 없다 · Create 는 없으면 만들고 있으면 참여 → 기동 순서 무관

`f_IpcMsgQSend()` ≡ `msgsnd()`

```
// ① 빈 슬롯 대기 ( 프로세스 간 세마포어 )
WaitForSingleObject(stpQ->vpSemEmpty, uiTimeOut_ms);

// ② 프로세스 간 상호배제 — 네임드 뮅텍스
WaitForSingleObject(stpQ->vpMutex, 5000U);

// ③ 공유메모리 위의 링버퍼에 write
stpRing->staSlot[stpRing->iTail] = *stpMsg;
stpRing->iTail = (stpRing->iTail + 1) % stpRing->iCapacity;
stpRing->nCount++;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemFull, 1, NULL); // ④ 찬 슬롯 +1
```

`f_IpcMsgQRecv()` ≡ `msgrcv()`

```
// ① 찬 슬롯 대기
WaitForSingleObject(stpQ->vpSemFull, uiTimeOut_ms);

// ② 같은 네임드 뮅텍스
WaitForSingleObject(stpQ->vpMutex, 5000U);

// ③ 링버퍼에서 read
*stpMsg = stpRing->staSlot[stpRing->iHead];
stpRing->iHead = (stpRing->iHead + 1) % stpRing->iCapacity;
stpRing->nCount--;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemEmpty, 1, NULL); // ④ 빈 슬롯 +1
```

실행 결과

[New Process] 로 창을 하나 더 띄우고 , 한쪽은 [Recv] · 다른쪽은 [Send] 를 누른다

● 프로세스 ① 수신 — [Recv] 를 먼저 눌러 큐를 만든다

```
14:22:05 [MQ] queue 'IpcDemoQ' open
14:22:05 [MQ] start (rx)
14:22:12 [MQ] rx 1
14:22:12 [MQ] rx 2
14:22:12 [MQ] rx 3
14:22:12 [MQ] rx 4
14:22:12 [MQ] rx 5
14:22:13 [MQ] rx 6
...
14:22:22 [MQ] rx 98
14:22:22 [MQ] rx 99
14:22:22 [MQ] rx 100
14:22:28 [MQ] rx done (100)
14:22:28 [MQ] stop
```

● 프로세스 ② 송신 — [New Process] 로 띄운 창에서 [Send]

```
14:22:12 [MQ] queue 'IpcDemoQ' open
14:22:12 [MQ] start (tx)
14:22:12 [MQ] tx 1
14:22:12 [MQ] tx 2
14:22:12 [MQ] tx 3
14:22:12 [MQ] tx 4
14:22:12 [MQ] tx 5
14:22:13 [MQ] tx 6
...
14:22:22 [MQ] tx 98
14:22:22 [MQ] tx 99
14:22:22 [MQ] tx 100
14:22:22 [MQ] tx done (100)
14:22:26 [MQ] stop
```

1

두 프로세스 사이로 100 건이 그대로 전달

송신 창의 tx 1~100 과 수신 창의 rx 1~100 이 정확히 일치한다 .

2

수신을 먼저 눌러도 , 송신을 먼저 눌러도 동작

Create 가 없으면 만들고 있으면 참여하므로 기동 순서에 의존하지 않는다 .

3

수신 창의 시각이 7 초 늦게 시작

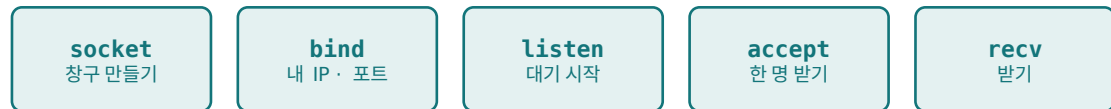
큐가 먼저 만들어져 대기하고 있다가 , 송신이 붙는 순간부터 받기 시작한다 — 비동기 동작 확인 .

TCP / IP 구현

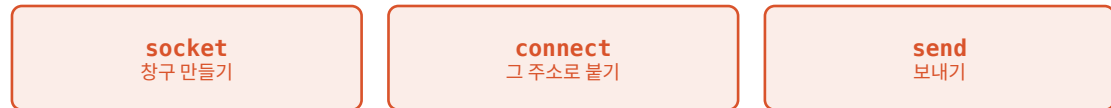
다른 PC · 다른 프로그램과의 송수신 — Linux SocketUtility v5.0 → Winsock2 포팅, 함수명 · 인자 · 상수는 원본 그대로 · lpcSocket.c

연결 맺는 순서 — 문을 열고 기다리는 쪽이 서버, 찾아가서 두드리는 쪽이 클라이언트

① 먼저 Rx · 수신 = 서버 [Recv] — 자리를 잡고 기다린다



② 그다음 Tx · 송신 = 클라이언트 [Send] — 그 자리를 찾아가다



※ accept 는 200ms 단위 select 루프, connect 는 논블로킹 + select — [Stop] 에 바로 반응한다

리눅스 원본을 Windows 로 옮기며 걸린 것 — 컴파일은 통과하는데 뜻이 달라지는 항목

항목	Linux	Windows
소켓 핸들	int	SOCKET (UINT_PTR) · x64 에서 64bit
송 / 수신 타임아웃	timeval 구조체	DWORD 밀리초 — 그대로 넘기면 엉뚱한 값
소켓 닫기	close()	closesocket()
초기화	없음	WSAStartup / WSACleanup 필요
에러 확인	errno	WSAGetLastError()
select 1 번 인자	maxfd + 1	무시된다 (있어도 되고 없어도 된다)

※ 타임아웃은 형이 맞아 보여 그냥 지나치기 쉽다 — 내부에서 timeval → DWORD ms 로 변환했다

f_SocketRecvTCP_IPv4() — TCP 는 바이트 스트림, 다 채울 때까지 반복

send(4 byte) 를 한 번 불러도 recv 는 2 byte 만 받을 수 있다 — 메시지 경계가 없다

```

// ① 요청한 크기를 다 채울 때까지 반복해서 받는다
while ((lRemain > 0) && (stpSocket->iSockStatus == Connected)) {
    iRecv = recv((SOCKET)stpSocket->hSockId, cpCursor, iChunk, 0);

    if (iRecv > 0) {
        // ② 받은 만큼 커서를 민다
        cpCursor += iRecv;
        lTotalRecv += iRecv;
        lRemain = lFixedDataSize - lTotalRecv;
    }
    else stpSocket->iSockStatus = Disconnected; // ③ 상대 종료 · 에러
}
    
```

f_TcpSenderProc() — 데모 송신, 프레임 헤더 없이 int 4 byte

```

// 1 부터 100 까지 0.1 초 간격으로 보낸다
for (iData = IPC_DEMO_FIRST_VALUE;
    iData <= IPC_DEMO_LAST_VALUE; iData++) {
    // htonl 체크박스가 켜져 있으면 네트워크 바이트 오더로 실는다
    iWire = (s_iDemoUseNB0 != 0) ? htonl(iData) : iData;
    lSent = f_SocketSendTCP_IPv4(&s_stDemoSocket, &iWire, sizeof(iWire));
    // 0.1 초 쉬면서 정지 요청도 같이 확인한다
    f_IsDemoStopRequested(IPC_DEMO_INTERVAL_MS);
}
    
```

경계는 앱이 정한다 — ① 길이 붙이기 ② 구분자 ③ 고정 길이 중 본 과제는 ③ 번 양쪽 다 little-endian 이라 그대로 맞고, 상대가 htonl 규약이면 체크박스를 켜다 보내는 쪽 f_SocketSendTCP_IPv4() 도 같은 모양 — 다 보낼 때까지 반복한다

실행 결과

한쪽은 [Recv](서버), 다른쪽은 [Send](클라이언트) — 다른 PC 의 프로그램과도 같은 방식으로 붙는다

● 프로세스 ① 수신 = 서버 — [Recv]

```
14:25:02 [TCP] start (rx)
14:25:02 [TCP] listen 0.0.0.0:51000
14:25:09 [TCP] accept 127.0.0.1:60312
14:25:09 [TCP] rx 1
14:25:09 [TCP] rx 2
14:25:09 [TCP] rx 3
14:25:10 [TCP] rx 4
14:25:10 [TCP] rx 5
...
14:25:19 [TCP] rx 98
14:25:19 [TCP] rx 99
14:25:19 [TCP] rx 100
14:25:20 [TCP] peer closed
14:25:20 [TCP] rx done (100)
```

● 프로세스 ② 송신 = 클라이언트 — [Send]

```
14:25:09 [TCP] start (tx)
14:25:09 [TCP] connecting 127.0.0.1:51000
14:25:09 [TCP] connected 127.0.0.1:51000
14:25:09 [TCP] tx 1
14:25:09 [TCP] tx 2
14:25:09 [TCP] tx 3
14:25:10 [TCP] tx 4
14:25:10 [TCP] tx 5
...
14:25:19 [TCP] tx 98
14:25:19 [TCP] tx 99
14:25:19 [TCP] tx 100
14:25:19 [TCP] tx done (100)
14:25:20 [TCP] stop
```

1 연결 → 100 건 → 정상 종료

accept 로 상대 IP · 포트를 확인하고 , 100 건을 받은 뒤 상대가 끊으면 peer closed 로 마무리된다 .

2 다른 PC 와 붙일 때

수신측 IP 를 0.0.0.0 으로 두고 방화벽에서 포트를 연다 . 상대가 htonl 을 쓰면 체크박스를 켜다 .

3 역할이 반대인 상대와도 가능

Tx / Rx Init 함수를 바꿔 부르기만 하면 된다 . 함수명 · 상수는 원본 SocketUtility 와 동일하게 두었다 .

정리

개념 세 가지와 , 그 개념이 코드에서 어떻게 만났는지

1

무텍스와 세마포어는 짝이다

상호배제는 무텍스 , 개수를 세고 기다리는 것은 세마포어 .
하나로 다른 하나를 흉내내면 바쁜 대기가 되거나 데이터가 깨진다 .

2

메시지 큐는 없으면 만들 수 있다

공유메모리 + 링버퍼 (FIFO) + 세마포어 2 개 + 무텍스 .
Windows 에 System V 메시지 큐가 없어 이 조합으로 msgsnd / msgrcv 와 같은 의미를 만들었다 .

3

포팅에서 위험한 것은 타입이 맞아떨어지는 차이다

SO_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을
먼저 찾아야 한다 . 소켓 핸들 크기 , 타임아웃 단위가 대표적이다 .

검증 결과 ToDo#1 · #2 · #3 모두 1 부터 100 까지 0.1 초 간격으로 송신하고 , 수신측이 같은 값을 빠짐없이 출력하는 것을 확인했다 .